Obligatory Assignment 1

Lars Haukland

March 2, 2025

Abstract

Creating a parallel radix sort and testing speedup.

1 Creating a sequential radix sort

My algorithm has been implemented using counting sort to count the number of occurrences for digits then sorting based on that and running this for each digit in the numbers. My pseudocode is as follows with b as the number of bits per digit and n the number of elements to be sorted.

```
arr = [nums]
count = [0 * 2^b]
output = []

for i in 0 .. 64 / b {
    digit_idx = i * b
    counting_sort(arr, n, digit_idx, count, output)
}

void counting_sort(arr, n, digit_idx, count, output) {
    set all counts = 0

    for i in 0..n {
        digit = (arr[i] >> digit_idx) & ((1ULL << b) - 1)
        count[digit] += 1
    }

    for i in 1..count.size() {
        count[i] += count[i-1]
    }

    for i in n-1..0 {
        digit = (arr[i] >> digit_idx) & ((1ULL << b) - 1)
        output[count[digit] - 1] = arr[i]
        count[digit] -= 1
    }

    swap(arr, output)
}
```

Regarding the runtime we first have to look at the runtime of counting sort. We have 4 loops and the max number of iterations for these loops are n or 2^b. The runtime of counting sort is $O(n + 2^b)$.

Then the outer loop runs $64/b$ times so the total running time is $O(64/b * (n + 2^b))$.

2 Experiments using sequential radix sort

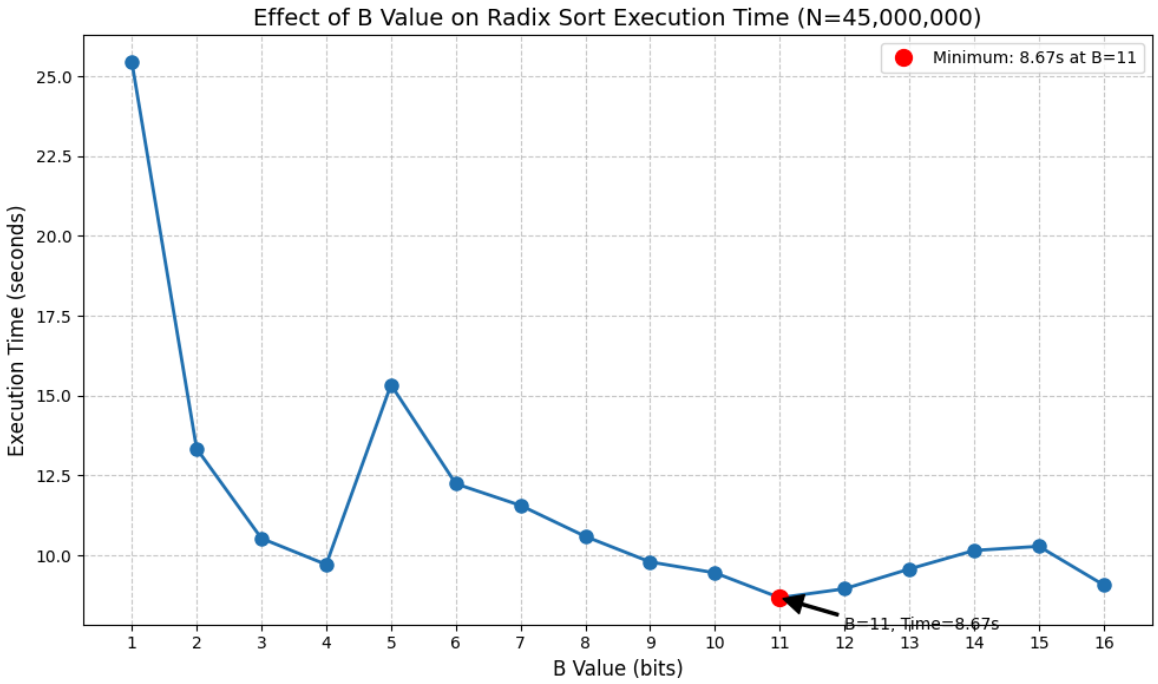
Here I perform experiments using the sequential radix sort and varying n and b to determine the largest n the program can sort in 10 seconds. Then when max n is found I test for different values of b. Then give timings for different parts of the program for some final n and b.

I think that the larger the b the fewer iterations my program will have. This is because of the outer loop running $O(64/b)$ iterations. So I thought if I want the largest n I should use the largest b possible. This would be 64. But then my program crashes, I spent some time thinking why but then I realized how much space this tried to allocate.

The count array needs to have length 2^b to be able to count all digits. And then the size of each element is an int which is 4 bytes. So with b = 64 the total space needed just for the count array is $4 * 2^{64}$) which is about 68 million Terabytes. Because of this both b = 64 and b = 32 are out of the question. Therefore I only test for $b \in [1 - 16]$.

The largest n that my sequential program could do with b = 16 was 45 million. Then my radix sort took a total of 9.07 seconds.

Here is a graph where I test different b values for n=45 million.



Quite suprisingly it was not b=16 that was best, but rather b = 11. I check with others that reported the same which was suprising. The following is timing for different parts of the program with n=45 million and b = 11.

Counting the number of elements in each bucket: 0.836 s.
Performing the prefix sum to determine the position of each bucket: 1.16131e-05 s.
Placing the elements in their correct bucket: 7.55 s.

I was surprised by how slow the last part is. Putting the elements in their correct position based on the prefix sum. The code for this part is:

```
for (int i = n - 1; i >= 0; i--) {
    unsigned long long digit = (arr[i] >> exp) & ((1ULL << b) - 1);
    output[count[digit] - 1] = arr[i];
    count[digit]--;
```

}

I don't think the digit calculation should take that long so my suspicion is that because we have to visit an array to get the index in output and then another array to get the value. I think this can take long if the arrays are stored far from each other in the array. Maybe I could have tried allocating these arrays close to each other.

In addition because we are updating the output array very spread out and not in a sequential manner. This may lead to cache misses where the part we are trying to update is not in cache.

3 Parallel radix sort

In my implementation of the parallel radix sort, I struggled a lot trying to parallelize the prefix sum. As it later turned out, this was not really necessary as the sequential version of prefix sum takes almost no time. The parts that are parallel are resetting, counting, and placing.

In this algorithm each thread has their own version of the count array. Then the numbers are evenly divided among threads and they count the digits in their own counting array. Then I have a 2d prefix array which is created sequentially to let each thread know where their digit should start to be placed.

Using this prefix array numbers are then evenly spread out on threads again and each thread uses their prefix array to place the number in the output array.

Lastly the output array and original array are swapped.

I also padded the counting array and prefix array in such a way that each value takes up the whole cache line. This prevents false sharing.

3.1 Pseudocode

```
arr = [nums]
count = [[0 * 2^b] * num_threads]
prefix = [[0 * 2^b] * num_threads]
output = [0 * n]

for i in 0 .. 64 / b {
    digit_idx = i * b
    counting_sort(arr, n, digit_idx, count, output)
}

void counting_sort(arr, n, digit_idx, count, output) {
#pragma omp parallel
{
    let thread_id = omp_get_thread_num()
    set all counts = 0

    #pragma omp for
    for i in 0..n {
        digit = (arr[i] >> digit_idx) & ((1ULL << b) - 1)
        count[thread_id][digit] += 1
    }

    #pragma omp single
    {
    let running_sum = 0
    for i in 0..2^b {
```

```
        for thread in 0..thread_count {
            prefix[thread][i] = running_sum
            running_sum += count[thread][i]
        }
    }
    }

    #pragma omp for
    for i in n-1..0 {
        val = arr[i]
        digit = (val >> digit_idx) & ((1ULL << b) - 1)
        output[prefix[thread_id][digit]++] = val
    }

    swap(arr, output)
}
}
```

3.2 Runtime

The runtime of this algorithm is, of course, a little harder to see at first glance. We again have the outer loop running $64/b$ times and then we have the counting sort function. This has three parts contributing to the runtime. Counting, prefix sum and placing. The counting evenly divides the numbers on the number of threads. So this has a runtime of $O(n/p)$.

The prefix sum is done in a single block so the number of threads do not affect this runtime. It is two loops, one running 2^b times and the inner one running p times. This makes the total running time $O(2^b * p)$.

Lastly the placing of elements is done. This has 1 for loop running n times and evenly spreads n numbers among p threads leading to a runtime of $O(n/p)$.

Total running time is then $O(64/b * (n/p + 2^b * p))$.

4 Scaling experiments

4.1 Strong scaling

For the strong scaling experiments I used, as wanted the n and b values from section 2. n=45 million and b = 11. First the speedup! Figure 1. Here its clear that p=35 gives the most speedup for n=45 million.

The following is a graph showing the total runtime of the sorting when p is ranging from [1-40] with a step of 5. See figure 1

A little suprising but still an ok result is that with 35 threads I got the best running time of 0.83 seconds. I also timed the different parts of the program. Here are some graphs for each part when p changes. See figures 2-4

As is clear and quite interesting from these plots the counting and placing do scale but the counting does not scale very well. My theory is that this may arise from cache misses or maybe true sharing in some way. Its also clear that as expected the prefix sum scales the wrong way. This is because the runtime is $O(2^b * p)$.

All in all, I think I could have gotten better scaling if the counting part scaled better. But I am still happy with this result.

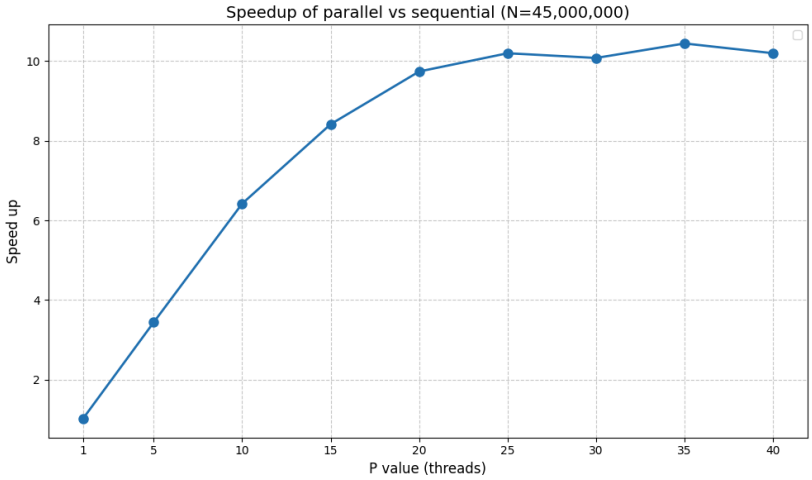


Figure 1: Speedup

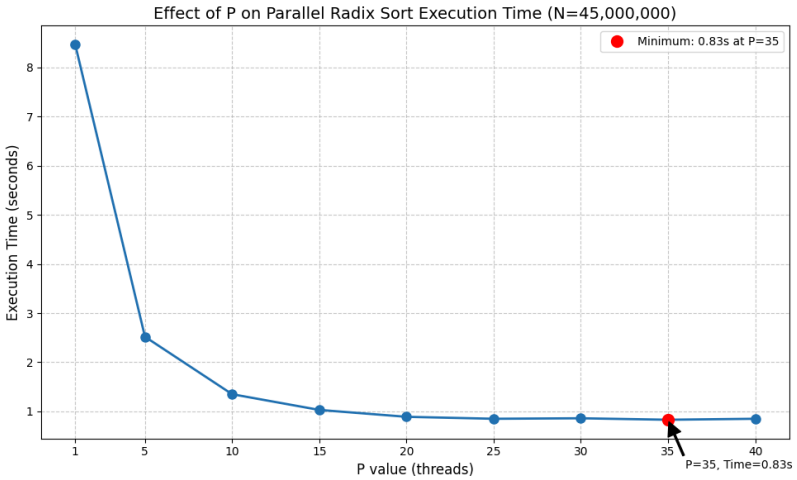


Figure 2: Strong scaling

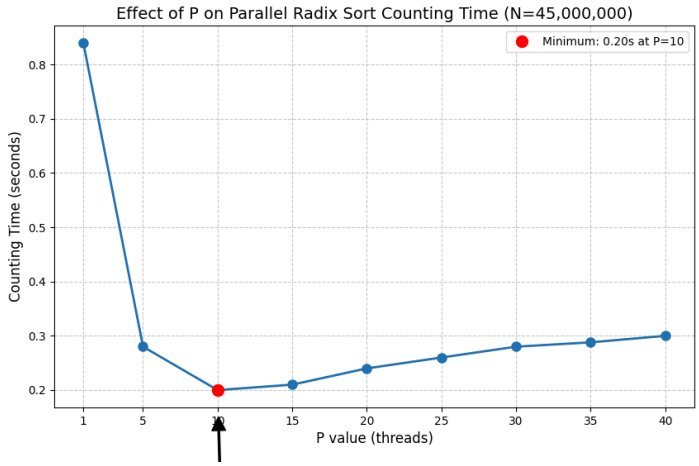


Figure 3: Strong scaling: counting

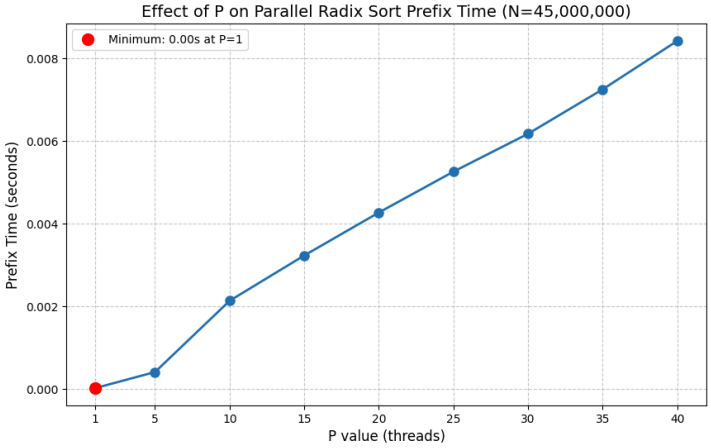


Figure 4: Strong scaling: prefix

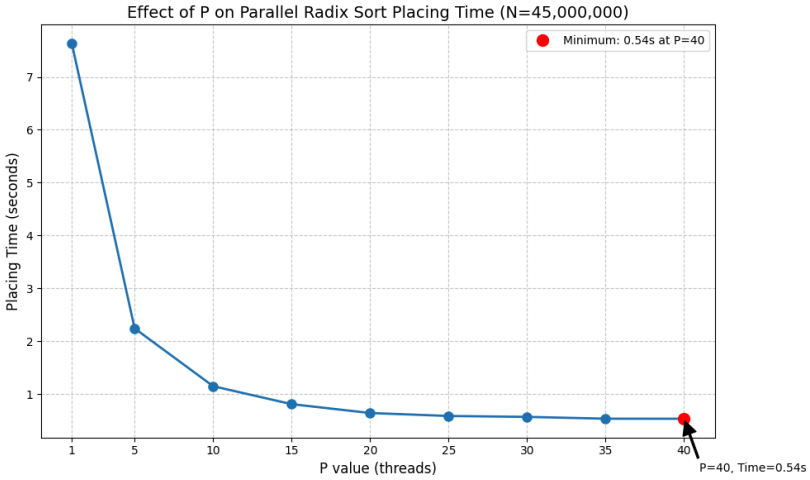


Figure 5: Strong scaling: placing

4.2 Weak scaling

See figure 5. As expected I dont get perfect weak scaling, if so it would be a straight line. We already know that since the prefix sum is not parallel this will always increase but it is such a small portion meaning that the counting and placing loops must be increasing. What is suprising is how when p=10 and 30 we get a sharp decrease in time. This can be because I'm doing these experiments on sunday, the last day of the assignment and brake is busy. Or it means that somehow the data lined up better with these amount of threads. It's hard to say but still very interesting.

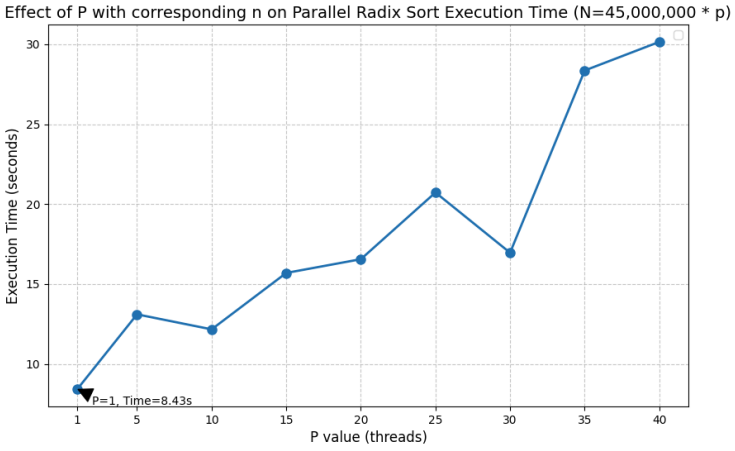


Figure 6: Weak scaling